What is VS Code

Visual Studio Code is a free code editor used for writing programs in JavaScript, Python, Node.js and more

---

Windows

Option 1 Microsoft Store (easy method)

Step 1 Open Microsoft Store
Step 2 Search Visual Studio Code
Step 3 Click Install
Step 4 Wait for installation
Step 5 Open VS Code from Start menu

Option 2 Official website (recommended)

Step 1 Go to https://code.visualstudio.com
Step 2 Download Windows installer
Step 3 Run setup file
Step 4 Click Next Next Install
Step 5 Launch VS Code

After install check

Open terminal in VS Code

Run:
node -v
npm -v

Linux (Ubuntu Debian)

Option 1 Snap package (easy)

Step 1 Open terminal
Step 2 Run:

sudo snap install code --classic

Step 3 Launch:

code

Option 2 Official website

Step 1 Go to https://code.visualstudio.com
Step 2 Download .deb file
Step 3 Install:

sudo dpkg -i code*.deb
sudo apt install -f

Step 4 Run:

code

---

macOS

Option 1 Official website

Step 1 Go to https://code.visualstudio.com
Step 2 Download macOS version
Step 3 Open zip file
Step 4 Drag Visual Studio Code into Applications folder
Step 5 Open from Applications

Option 2 Homebrew

Step 1 Install Homebrew if not installed

/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

Step 2 Install VS Code

brew install --cask visual-studio-code

Step 3 Run

code

First setup after install

Step 1 Open VS Code
Step 2 Go to Extensions
Step 3 Install:
Node.js extension support (optional)
Python extension (if needed)

Verify working

Open terminal in VS Code and run:

node -v
npm -v

Common issues

Windows Microsoft Store not opening use website install
Linux command not found reinstall snap or dpkg
macOS security blocked go to System settings allow app

• Install Node.js and npm on Windows

- Step 1: Download
  Go to https://nodejs.org
- Click **Get Node.js**
- Click windows installer(.msi)- it download the LTS version
- Step 2: Install
  Open the downloaded file
  Click next - next
  Accept terms
  Make sure “Add to PATH” is selected
- (Don’t checked any other box)
  Click Install
- Step 3: Check installation
  Open PowerShell
  Run:
  node -v
  npm -v
- If versions appear, installation is successful
- ---

• Install Node.js and npm on Linux (Ubuntu/Debian)

- Step 1: Update system
  sudo apt update
- Step 2: Install Node.js and npm
  sudo apt install nodejs npm -y
- Step 3: Check installation
  node -v
  npm -v
- Optional better method for latest version:
  curl -fsSL https://deb.nodesource.com/setup_lts.x | sudo -E bash -
  sudo apt install -y nodejs
- ---

• Install Node.js and npm on macOS

- Option 1: Installer
  Go to https://nodejs.org
  Download LTS macOS installer (.pkg)
  Open and install normally
- Option 2: Using Homebrew
- Install Homebrew:
  /bin/bash -c "$(curl -fsSL
  https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
- Install Node:
  brew install node
- Check:
  node -v
  npm -v
- ---

• Test Node.js

- Run:
  node -e "console.log('Node is working')"
- ---

• Common issues

- If node is not recognized on Windows
  Restart system after install
  OR check PATH includes Node.js folder **OR** open powershell as administrator and run **`Set-ExecutionPolicy RemoteSigned -Force`**
- If permission error on Linux or macOS
  Use sudo before commands
- ---

What you are installing

1. MongoDB Server (database engine)
2. MongoDB Shell (mongosh) (command line)
3. MongoDB Compass (GUI tool)

Windows

Step 1: Download MongoDB (Server + Shell)

Go to:
https://www.mongodb.com/try/download/community

Download:

- Windows
- MSI package
- Install “Complete”

During install:

- Check “Install MongoDB as a Service”
- This installs server automatically

Step 2: Install MongoDB Shell (mongosh)

Go to:
https://www.mongodb.com/try/download/shell

Select platform - windows x64 (10+) and package- msi

Download Windows version
Install normally

Test:
mongosh

Step 3: Install MongoDB Compass (GUI)

Go to:
https://www.mongodb.com/try/download/compass

Select platform - windows 64bit(10+) (MSI)

Download Windows version
Install normally

Open Compass and connect using:
mongodb://localhost:27017

Step 4: Test everything

Open Command Prompt:

mongosh
show dbs

Linux (Ubuntu/Debian)

Step 1: Install MongoDB Server

sudo apt update

sudo apt install -y mongodb-org

Step 2: Start MongoDB

sudo systemctl start mongod
sudo systemctl enable mongod

Check:
sudo systemctl status mongod

Step 3: Install MongoDB Shell

Install shell:

curl -fsSL https://pgp.mongodb.com/server-7.0.asc | sudo gpg -o /usr/share/keyrings/mongodb.gpg --dearmor

echo "deb [signed-by=/usr/share/keyrings/mongodb.gpg] https://repo.mongodb.org/apt/ubuntu jammy/mongodb-org/7.0 multiverse" | sudo tee /etc/apt/sources.list.d/mongodb-org.list

sudo apt update
sudo apt install -y mongodb-mongosh

Run:
mongosh

Step 4: Install MongoDB Compass

Go to:
https://www.mongodb.com/try/download/compass

Download Linux .deb file

Install:
sudo dpkg -i mongodb-compass*.deb
sudo apt-get install -f

Open Compass:
mongodb://localhost:27017

---

macOS

Step 1: Install MongoDB Server (via Homebrew)

Install Homebrew if not installed:
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

Then:

brew tap mongodb/brew
brew install mongodb-community@7.0

Start:
brew services start mongodb-community@7.0

Step 2: Install MongoDB Shell

brew install mongosh

Run:
mongosh

Step 3: Install MongoDB Compass

Go to:
https://www.mongodb.com/try/download/compass

Download macOS version
Install normally

Open and connect:
mongodb://localhost:27017

Test (All OS)

Run:

mongosh

Then:

show dbs
use test
db.users.insertOne({name: "demo"})

Connection string for Compass

Use:
mongodb://localhost:27017

If something fails

Windows:
Restart system after install

Linux:
Check service:
sudo systemctl status mongod

macOS:
brew services restart mongodb-community@7.0

Set PATH

Windows
PATH usually needs to be added manually

Linux
No need if installed properly

macOS
No need if using Homebrew

Windows

Step 1 Find MongoDB path

Usually installed at:

C:\Program Files\MongoDB\Server\7.0\bin

Step 2 Add to PATH

1. Press Windows key
2. Search Environment Variables
3. Click Edit the system environment variables
4. Click Environment Variables
5. Under System variables find Path → click Edit
6. Click New
7. Paste:

C:\Program Files\MongoDB\Server\7.0\bin

8. Click OK and save

Step 3 Restart terminal

Close Command Prompt or PowerShell and open again

Step 4 Test

Run:

mongod
mongosh

Run mongodb on-

Windows

Method 1 (Normal install – service mode)

If you installed MongoDB as a service:

Start MongoDB

It runs automatically after install.

If not running:

1. Press Windows key

2. Search “Services”
3. Find **MongoDB Server**
4. Click Start

Check if running

Open Command Prompt:

mongosh

Then:

show dbs

Method 2 (manual start)

Open Command Prompt as Admin:

net start MongoDB

Stop:

net stop MongoDB

Start MongoDB manually (if needed)

If no service exists:

mongod

(Default data path is used)

Linux (Ubuntu/Debian)

Start MongoDB

sudo systemctl start mongod

Enable auto start

sudo systemctl enable mongod

Check status

sudo systemctl status mongod

Open MongoDB shell

mongosh

Stop MongoDB

sudo systemctl stop mongod

---

macOS

If installed via Homebrew

Start MongoDB

brew services start mongodb-community@7.0

Stop MongoDB

brew services stop mongodb-community@7.0

Restart MongoDB

brew services restart mongodb-community@7.0

Open shell

mongosh

Test MongoDB (all OS)

After opening shell:

show dbs

Create test DB:

use mydb

db.users.insertOne({name: "test"})

db.users.find()

MongoDB Compass (GUI)

Not a server — just UI.

To use:

1. Start MongoDB server first
2. Open Compass
3. Connect using:

mongodb://localhost:27017

Common problems

Windows:

- mongosh not found → reinstall shell or fix PATH
- service not running → start from Services

Linux:

- “connection refused” → MongoDB not started

macOS:

- service not starting → run brew services restart

---

summary

Windows:
Start service or run mongod

Linux:
sudo systemctl start mongod

macOS:
brew services start mongodb-community

What is Postman

Postman is a tool used to test APIs like GET POST PUT DELETE requests

---

Windows

Step 1 Download
Go to https://www.postman.com/downloads

Download Windows version

Step 2 Install
Open downloaded file
It will install automatically
Postman will open after install

Step 3 Login or skip
You can sign in or click skip

Step 4 Test
Open Postman
Enter URL like
https://jsonplaceholder.typicode.com/posts
Click Send

Linux Ubuntu Debian

Step 1 Download
Go to https://www.postman.com/downloads
Download Linux 64 bit file tar gz

Step 2 Extract
Open terminal

tar -xvzf Postman-linux-x64.tar.gz

Step 3 Move to opt folder

sudo mv Postman /opt

Step 4 Create shortcut

sudo ln -s /opt/Postman/Postman /usr/bin/postman

Step 5 Run Postman

postman

---

macOS

Step 1 Download
Go to https://www.postman.com/downloads
Download macOS version

Step 2 Install
Open dmg file
Drag Postman into Applications folder

Step 3 Run
Open Applications
Click Postman
Login or skip

How to use Postman

Step 1 Open Postman
Step 2 Select request type GET POST etc
Step 3 Enter API URL
Step 4 Click Send
Step 5 See response below

Example test API

https://jsonplaceholder.typicode.com/posts

Method GET

Click Send to see data

Common issues

Windows not opening reinstall Postman
Linux command not found check symlink
macOS not opening allow in security settings

MERN website URL

https://mern-five-topaz.vercel.app/

Free Online Drawing Tool - Sketch & Paint Online | DrawIsland

mongod
 starts database server

mongosh
 connects and runs queries

MongoDB Document
MongoDB CRUD Operations - Database Manual - MongoDB Docs

MongoDB expressions/operators/accumulators
Expressions - Database Manual - MongoDB Docs

Binary = raw bytes (no meaning by itself)
BSON = binary **with structure and labels**

admin → security & control
local → server-only data
config → system metadata

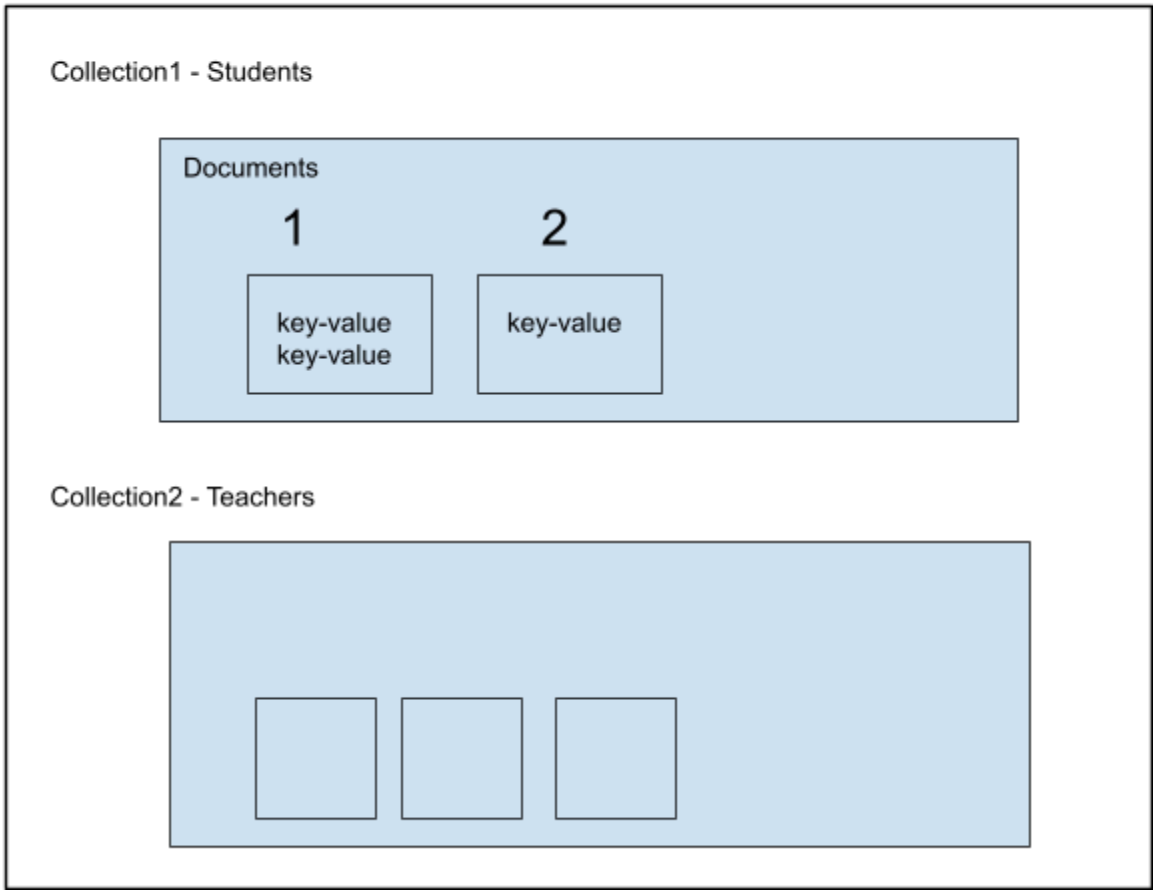
${server.address().port} for power 0 in express.js

MERN Stack Website URL

https://mern-five-topaz.vercel.app/

MongoDB

DB - school



Js-Node-React (Similarity)

Feature	JavaScript	Node.js	React
Language used	JavaScript	JavaScript	JavaScript
Used in web	Yes	Yes	Yes
Purpose	Build web apps	Build web apps	Build web apps
Works together	Yes	Yes	Yes
Community support	Large	Large	Large
Open source	Yes	Yes	Yes
Easy to start	Yes	Yes	Yes
Used by companies	Yes	Yes	Yes
Regular updates	Yes	Yes	Yes
Cross platform	Yes	Yes	Yes
Popular in industry	Yes	Yes	Yes
Used by developers	Yes	Yes	Yes

JavaScript & Node.js are ~70–80% similar

JavaScript and Node.js have the same syntax, but different uses (frontend vs backend)

Js-Node-React (Difference)

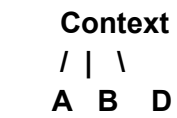
Feature	JavaScript	Node.js	React
Type	Programming language	Runtime environment	Library
Runs where	Browser	Server	Browser
Main role	Write logic	Build backend	Build user interface
Backend support	No	Yes	No
UI support	Limited	No	Yes
Installation	Not needed in browser	Needs installation	Needs setup
Use case	Basic scripting	APIs and server apps	Interactive web apps
Performance use	Light tasks	Handles many users	Fast UI rendering

Node - Express

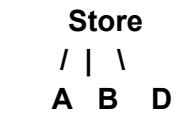
Feature	package.json	package-lock.json	node_modules/
What is it?	The Manifest (The Menu)	The Blueprint (The Receipt)	The Warehouse (The Ingredients)
Content	Lists metadata, scripts, and version ranges (e.g., `^4.18.2`).	Lists the exact version and dependency tree of every package.	The actual source code of the libraries you installed.
Purpose	To define what your project needs to run.	To ensure every developer gets the exact same version.	To provide the logic your code calls (e.g., require('express')).
Editable?	Yes, you edit this to add or remove packages.	No, generated automatically by npm.	No, never edit these files directly.
Git?	Commit to GitHub.	Commit to GitHub.	Ignore (add to `.gitignore`).

Prop Drilling:
App → A → B → C → D

Context:



Redux:



Feature	Prop Drilling	Context API	Redux
What it is	Passing data via props through many components	Built-in way to share global data	External library for state management
Setup	No setup	Easy	Medium (needs setup)
Best for	Small apps	Medium apps	Large apps
Data flow	Parent → Child → Deep Child	Direct access from anywhere	Central store (single source of truth)
Code complexity	Low (initially)	Medium	High
Scalability	Poor	Medium	Excellent
Performance	OK for small	Can re-render many components	Optimized (with tools)
Reusability	Low	Medium	High
Debugging	Hard (deep props)	Medium	Easy (DevTools)
Real use cases	Passing simple props	Theme, Auth, Language	Cart, Dashboard, Complex apps

Feature	Library	Package	Dependency
What is it?	A collection of pre-written code/functions.	A bundled unit of code with a version and name.	A relationship; code that your project *depends* on.
Analogy	A Book (Information you can use).	A Boxed Set (The book, the cover, and the barcode).	An Item on a List (What you need to finish a job).
Format	Pure logic/code (e.g., a `.js` file).	A folder containing code + a `package.json` file.	Entry inside your `package.json` file.
Focus	Functionality: "What can this code do for me?"	Distribution: "How do I download/install this?"	Requirement: "My project won't run without this."
Example	React, Lodash, Axios.	The `express` folder inside your `node_modules`.	`"express": "^4.18.2"` in your manifest.

Feature	Vitest	Jest	Supertest	Playwright
Primary Level	Unit & Integration	Unit & Integration	Integration (API)	E2E (End-to-End)
Environment	Node.js (Simulated DOM)	Node.js (Simulated DOM)	Node.js (HTTP)	Real Browsers (Chrome, Safari, Firefox)
Speed	Instant (Fastest)	Fast	Moderate	Slow (High overhead)
Testing Goal	Individual functions or UI components.	Individual functions or UI components.	Verifies API endpoints and server logic.	Verifies the entire user journey (Login -> Buy -> Logout).
What it sees	Code logic / Variables.	Code logic / Variables.	JSON / HTTP Status codes.	The UI, buttons, and visual layout.

Day 1 Notes - MongoDB Intro and Basics

MERN stands for:
MongoDB - that's the database, NoSQL type, stores data in flexible documents like JSON.
Express.js - framework for the backend, works with Node.js.
React - library for building the user interface on the front end.
Node.js - lets you run JavaScript on the server side.

How the whole thing flows:
The user on client (React) makes a request, like clicking something.
Server (Node/Express) gets the request.
Server talks to database (MongoDB) to get data.
DB sends data back to the server.
Server sends response to client.
Client updates the UI.

For setup, need to install:
Visual Studio Code - code editor, get from code.visualstudio.com.
Node.js and NPM - download from nodejs.org, NPM comes with it.
MongoDB Community Server - from mongodb.com.
Postman - for testing APIs, postman.com.

After install, check versions:
code --version
node -v
npm -v
mongod --version

Now MongoDB basics.
SQL vs NoSQL:
SQL databases like MySQL have tables with fixed columns, use SQL queries, good for structured data.
NoSQL like MongoDB has flexible documents, no fixed schema, better for changing data or big scale.

Example queries:
SQL: SELECT * FROM users WHERE age > 25;
Mongo: db.users.find({ age: { $gt: 25 } })

CRUD operations:
Create - insertOne({name: "John", age: 30}) for one doc,
insertMany([{name: "Jane"}, {name: "Bob"}]) for multiple.

Read - find() for all docs,

find({age: {$gt: 25}}) for filtered,
findOne({name: "John"}) for first match.

Update - updateOne({name: "John"},{$set: {age: 31}}) for one,
updateMany({age: {$lt: 30}}, {$set: {status: "young"}}) for many.

Delete - deleteOne({name: "John"}) for one,
deleteMany({age: {$gt: 60}}) for many.

Count - countDocuments() for total,
countDocuments({age: {$gt: 25}}) for filtered count.

Replace - replaceOne({name: "John"},
{name: "Johnny", age: 31}) to replace whole doc.

To start: show dbs to list, use db_name to switch, db.createCollection("name") to make a collection.

Day 1 Assignment - MongoDB CRUD Operations

Question 1: Animal Collection CRUD
1. Create a collection called "animals" and insert the following documents:
   - { name: "Lion", type: "Mammal", habitat: "Savanna" }
   - { name: "Eagle", type: "Bird", habitat: "Mountains" }
   - { name: "Shark", type: "Fish", habitat: "Ocean" }

2. Find all animals in the collection.

3. Find animals where type is "Mammal".

4. Update the Lion's habitat to "Grassland".

5. Delete the Shark document.

6. Count the remaining documents in the animals collection.

Question 2: Plant Collection CRUD
1. Create a collection called "plants" and insert the following documents:
   - { name: "Rose", type: "Flower", color: "Red" }
   - { name: "Oak", type: "Tree", color: "Green" }
   - { name: "Cactus", type: "Succulent", color: "Green" }

2. Find all plants in the collection.

3. Find plants where color is "Green".

4. Update the Rose's color to "Pink".

5. Delete the Cactus document.

6. Count the remaining documents in the plants collection.

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Exam> mongosh
Current Mongosh Log ID: 69ccf17b1abe535d1a3682d0
Connecting to:
mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.2
Using MongoDB: 8.2.6
Using Mongosh: 2.8.2

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

The server generated these startup warnings when booting
2026-04-01T14:39:34.609+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
---

test> show dbs
school> show dbs
admin 40.00 KiB
config 96.00 KiB
local 40.00 KiB
school> exit
PS C:\Users\Exam> mongosh
Current Mongosh Log ID: 69ccf55b2ac43646043682d0
Connecting to:
mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.2
Using MongoDB: 8.2.6
Using Mongosh: 2.8.2

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/
school> show collections

school> db.createCollection("students")
{ ok: 1 }
school> show dbs
admin 40.00 KiB
config 108.00 KiB
school> db.students.updateOne({"stud_id":"05"},{$set:{"stud_id":"050"}})

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
school> db.students.find()
[
  {
    _id: ObjectId('69ccfa082ac43646043682d1'),
    stud_id: '050',
    name: 'Himanshu',
    city: 'Washim',
    age: 25,
    course: 'FBD',
    mobile: '789456130'
  }
]
school> db.students.updateOne({stud_id:"05"},{$set:{stud_id:"60"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 0
}
school> db.students.find()
[
  {
    _id: ObjectId('69ccfa082ac43646043682d1'),
    stud_id: '050',
    name: 'Himanshu',
school> db.students.insertOne({"stud_id":"08", name: "Sachin", city:"patna", age:25, course:
"FBD", mobile:"789456130"})
{
  acknowledged: true,
  insertedId: ObjectId('69cd00722ac43646043682d2')
}
school> db.students.find()
[
  {
    _id: ObjectId('69ccfa082ac43646043682d1'),
    stud_id: '60',
```

```
    name: 'Himanshu',
    city: 'Washim',
    age: 25,
    course: 'FBD',
    mobile: '789456130'
  },
  {
    _id: ObjectId('69cd00722ac43646043682d2'),
    stud_id: '08',
    name: 'Sachin',
    city: 'patna',
    age: 25,
    course: 'FBD',
    mobile: '789456130'
  }
]
school> db.students.updateOne({stud_id:"60"},{$set:{name:"Payal"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
school> db.students.updateOne({course:"FBD"},{$set:{name:"Payal"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 0,
  upsertedCount: 0
}
school> db.students.find()
[
  {
    _id: ObjectId('69ccfa082ac43646043682d1'),
    stud_id: '60',
    name: 'Payal',
    city: 'Washim',
    age: 25,
    course: 'FBD',
    mobile: '789456130'
  },
  {
```

```
    _id: ObjectId('69cd00722ac43646043682d2'),
    stud_id: '08',
    name: 'Sachin',
    city: 'patna',
    age: 25,
    course: 'FBD',
    mobile: '789456130'
  }
]
school> db.students.updateMany({course:"FBD"},{$set:{name:"Payal"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 1,
  upsertedCount: 0
}
school> db.students.find()
[
  {
    _id: ObjectId('69ccfa082ac43646043682d1'),
    stud_id: '60',
    name: 'Payal',
    city: 'Washim',
    age: 25,
    course: 'FBD',
    mobile: '789456130'
  },
  {
    _id: ObjectId('69cd00722ac43646043682d2'),
    stud_id: '08',
    name: 'Payal',
    city: 'patna',
    age: 25,
    course: 'FBD',
    mobile: '789456130'
  }
]
school> db.students.find({stud_id:"60"})
[
  {
    _id: ObjectId('69ccfa082ac43646043682d1'),
    stud_id: '60',
    name: 'Payal',
```

```
    city: 'Washim',
    age: 25,
    course: 'FBD',
    mobile: '789456130'
  }
]
school> db.students.find({stud_id:60})

school> db.students.insertOne({"stud_id":"08", name: "Sachin"})
{
  acknowledged: true,
  insertedId: ObjectId('69cd032c2ac43646043682d3')
}
school> db.students.insertMany({"stud_id":"08", name: "Sachin"},{name:"payal", age: 23},
{name: "aryan", age: 23})
MongoInvalidArgumentError: Argument "docs" must be an array of documents
school> db.students.insertMany([{"stud_id":"08", name: "Sachin"},{name:"payal", age: 23},
{name: "aryan",age:23}])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69cd04202ac43646043682d4'),
    '1': ObjectId('69cd04202ac43646043682d5'),
    '2': ObjectId('69cd04202ac43646043682d6')
  }
}
school> db.students.insertMany([{},{},{}])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69cd043e2ac43646043682d7'),
    '1': ObjectId('69cd043e2ac43646043682d8'),
    '2': ObjectId('69cd043e2ac43646043682d9')
  }
}
school> db.students.find()
[
  {
    _id: ObjectId('69ccfa082ac43646043682d1'),
    stud_id: '60',
    name: 'Payal',
    city: 'Washim',
    age: 25,
    course: 'FBD',
```

```
    mobile: '789456130'
  },
  {
    _id: ObjectId('69cd00722ac43646043682d2'),
    stud_id: '08',
    name: 'Payal',
    city: 'patna',
    age: 25,
    course: 'FBD',
    mobile: '789456130'
  },
  {
    _id: ObjectId('69cd032c2ac43646043682d3'),
    stud_id: '08',
    name: 'Sachin'
  },
  {
    _id: ObjectId('69cd04202ac43646043682d4'),
    stud_id: '08',
    name: 'Sachin'
  },
  { _id: ObjectId('69cd04202ac43646043682d5'), name: 'payal', age: 23 },
  { _id: ObjectId('69cd04202ac43646043682d6'), name: 'aryan', age: 23 },
  { _id: ObjectId('69cd043e2ac43646043682d7') },
  { _id: ObjectId('69cd043e2ac43646043682d8') },
  { _id: ObjectId('69cd043e2ac43646043682d9') }
]
school> db.students.deleteOne({stud_id:60})
{ acknowledged: true, deletedCount: 0 }
school> db.students.deleteOne({stud_id:"60"})
{ acknowledged: true, deletedCount: 1 }
school> db.students.find()
[
  {
    _id: ObjectId('69cd00722ac43646043682d2'),
    stud_id: '08',
    name: 'Payal',
    city: 'patna',
    age: 25,
    course: 'FBD',
    mobile: '789456130'
  },
  {
    _id: ObjectId('69cd032c2ac43646043682d3'),
```

```
    stud_id: '08',
    name: 'Sachin'
  },
  {
    _id: ObjectId('69cd04202ac43646043682d4'),
    stud_id: '08',
    name: 'Sachin'
  },
  { _id: ObjectId('69cd04202ac43646043682d5'), name: 'payal', age: 23 },
  { _id: ObjectId('69cd04202ac43646043682d6'), name: 'aryan', age: 23 },
  { _id: ObjectId('69cd043e2ac43646043682d7') },
  { _id: ObjectId('69cd043e2ac43646043682d8') },
  { _id: ObjectId('69cd043e2ac43646043682d9') }
]
school> db.students.countDocuments()
8
```

Day 2 Notes -
MongoDB Advanced Operations

Advanced CRUD and Aggregation

BSON and Data Types

MongoDB doesn't store data as plain text JSON. It uses BSON (Binary JSON).

Why BSON? It's a binary format that is faster for computers to parse and supports more data types than JSON.

Key Data Types:

ObjectId: A unique 12 byte ID automatically generated for each document. Acts as the primary key.
String: Standard text.
Boolean: true or false.
Date: Stores the current date and time.
Int32 and Int64: For whole numbers of different sizes.
Double: For floating point (decimal) numbers.
BinData: For storing binary data like a small image or a file directly inside a document (must be less than 16MB).
Embedded Object: A document nested inside another document. Example: { user: { name: "John", age: 30 } }.
Array: A list of values, which can be of any data type.

Advanced CRUD Lab Concepts

Query Operators: Used for more complex filtering.
$gt: Greater Than. Example: db.employees.find({ experience: { $gt: 5 } })
$lt: Less Than. Example: db.employees.find({ experience: { $lt: 1 } })

Update Operators:
$set: Sets the value of a field.
$mul: Multiply. Used to multiply the value of a field by a number. Great for percentage increases. Example: db.employees.updateOne({ name: "Grace" }, { $mul: { salary: 1.1 } }) (Gives a 10% raise).

Projection:
This means selecting which fields to include or exclude in your query results.
1 means include the field.
0 means exclude the field.

Example: db.employees.find({}, { name: 1, salary: 1, _id: 0 }) will show only the name and salary, and hide the default _id field.

Sort:
Used to order the results.
1 means sort in ascending order (A to Z, 1 to 10).
-1 means sort in descending order (Z to A, 10 to 1).
Example: db.employees.find().sort({ experience: -1 })

Practical Example Queries:
Read: Find employees in the Engineering department
db.employees.find({ department: "Engineering" })

Update: Increase salary by 10% for employees with more than 5 years of experience
db.employees.updateMany({ experience: { $gt: 5 } }, { $mul: { salary: 1.1 } })

Delete: Remove employees with less than 1 year of experience
db.employees.deleteMany({ experience: { $lt: 1 } })

Advanced Query with Condition, Projection, and Sort:
db.employees.find({ department: "Engineering" }, { name: 1, salary: 1, _id: 0 }).sort({ experience: -1 })

Aggregation Pipeline

What it is: A powerful tool for performing multi stage data processing and analysis. Think of it as a factory assembly line for your data.
How it works: You pass documents through a series of stages. The output of one stage becomes the input for the next.
Syntax: db.collection.aggregate([{ stage_1 }, { stage_2 }, ...])

Most Common Stages:
$match: Filters documents. (Like WHERE in SQL). Performance Tip: Use $match as early as possible!
$group: Groups documents by a key and calculates aggregate values (like sum, average, count). (Like GROUP BY in SQL).
$sort: Sorts the output documents. (Like ORDER BY in SQL).
$project: Reshapes documents: selects, renames, or removes fields. (Like SELECT in SQL).
$unwind: Deconstructs an array field. If a document has an array of 3 items, $unwind will create 3 separate documents.
$lookup: Performs a left outer join to another collection to combine data. (Like LEFT JOIN in SQL).

Example Queries for Stages:

$match Example:
db.employees.aggregate([{ $match: { department: "Engineering" } }])

$group Example:
db.employees.aggregate([{ $group: { _id: null, averageSalary: { $avg: "$salary" } } }])

$sort and **$limit** Example:
db.employees.aggregate([{ $sort: { salary: -1 } }, { $limit: 3 }])

$unwind Example:
db.employees.aggregate([{ $unwind: "$skills" }])

Multi stage pipeline Example (Match, Group, Sort):
```
db.employees.aggregate([
  { $match: { department: "Engineering" } },
  { $group: { _id: "$department", averageSalary: { $avg: "$salary" } } },
  { $sort: { averageSalary: -1 } }
])
```

Scalability and Advanced Concepts

GridFS: Storing Large Files

Problem: MongoDB documents have a size limit of 16 MB. You can't store large files (like videos or high res images) directly.
Solution: GridFS is MongoDB's system for storing files larger than 16 MB.
How it works: It breaks the large file into smaller chunks (e.g., 255 KB each) and stores these chunks in a dedicated collection (fs.chunks). A separate collection (fs.files) stores the metadata about the file (like the filename and type).

Replication: For High Availability

What it is: The process of keeping identical copies of your data across multiple servers. This group of servers is called a Replica Set.
How it works: One server is the Primary (receives all write operations). The other servers are Secondaries that copy data from the primary.
Why it's important: If the Primary server fails, a Secondary is automatically elected as the new Primary. This provides High Availability and prevents your application from going down.

Sharding: For Horizontal Scalability

What it is: The process of distributing data from a very large collection across multiple servers (or shards).

How it works: Each shard stores a piece of the total data. A special router called mongos directs application queries to the correct shard.
Why it's important: When your dataset becomes too large or has too much traffic for a single server to handle, sharding allows you to scale horizontally by adding more servers to share the load.

Day 2 Assignment - Data Modeling with Countries and States

Question 1: Countries Collection CRUD

1. Create a database called 'world_db'.
2. Create a collection called 'countries'.
3. Insert the following documents.

```
- {
    name: "USA",
    capital: "Washington, D.C.",
    states: [
      { name: "California", capital: "Sacramento" },
      { name: "Texas", capital: "Austin" }
    ]
  }
- {
    name: "India",
    capital: "New Delhi",
    states: [
      { name: "Maharashtra", capital: "Mumbai" },
      { name: "Karnataka", capital: "Bengaluru" },
      { name: "Tamil Nadu", capital: "Chennai" },
      { name: "Uttar Pradesh", capital: "Lucknow" },
      { name: "Rajasthan", capital: "Jaipur" }
    ]
  }
```

4. Find the document for "India".

5. Add a new state, { name: "Gujarat", capital: "Gandhinagar" }, to India's 'states' array.

6. Write a query to find only the states of India. Use a projection.

Question 2: Advanced Queries & Aggregation

1. Using the 'countries' collection, find the country that has a state named "Texas".

2. Write a query to pull the state "Karnataka" from the 'states' array in the India document.

3. Create an aggregation pipeline that does the following:
   - Unwinds the 'states' array.
   - Counts the total number of states listed across all countries in the collection.
   - Groups the result to show a single total count.

   Example pipeline structure:

```
[
  { $unwind: "$states" },
  { $count: "total_states_in_db" }
]
```

```
db.employees.updateMany({experience:{$gt:5}},{$mul:{salary:1.1}})

db.employees.deleteMany({experience: {$lt: 5} })

db.employees.find({dept:"development"},{name:1, salary:1}).sort({experience:-1})
db.employees.find({dept:"development"},{age:1}).sort({experience:-1})

db.employees.find().pretty()
db.employees.find({})

db.employees.aggregate([ ])

db.employees.aggregate([ {  $group: {  _id: null, avgSalary: { $avg: "$salary" }
db.employees.aggregate([ {  $group: {  _id: "$dept", avgSalary: { $avg: "$salary" }  }  } ])

db.employees.aggregate([{  $sort: { salary: -1 }  }])
db.employees.aggregate([{  $sort: {age:1}   }, { $limit: 2 }])

{ name: "Anik", skills: ["Java", "MongoDB"] }

{
  "emp_id": "19",
  "name": "Rehana",
  "dept": "Law",
  "salary": 50160.00000000001,
  "experience": 6,
  "age": 26,
  "skills": ["Java", "MongoDB"]
}
```

HTML, CSS, and JavaScript Fundamentals

HTML: The Structural Bones of the Web

What it is: HTML (HyperText Markup Language) defines the structure and content of a webpage.
Analogy: The blueprint, framing, and foundation of a house.

Key Concepts:
Elements and Tags: HTML uses tags to create elements. Most have opening and closing tags (e.g., text), some are self closing (e.g., <img src="image.jpg">).
Common Tags:
- html: The root element that wraps everything.
- head: Contains metadata, the page title, and links to CSS files.
- body: Contains all the visible content on the page.
- h1 to h6: Headings, where h1 is the largest and most important.
- p: Paragraphs of text.
- a: Anchor tags for links. It uses the "href" attribute to set the destination.
- div and span: Containers for grouping elements together. Div is a block (takes up the whole width), span is inline (only takes up the space it needs).
- input, button: Form controls to allow user interaction.
Attributes: Provide extra information about elements. Common ones include "id" (a unique identifier), "class" (a group identifier for styling), and "src" (the source file for an image).

HTML Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>FinTech Dashboard</title>
</head>
<body>
  <div id="header">
    <h1>Welcome to Your Account</h1>
  </div>
  <div class="content">
    <p>Your current balance is $5,000.</p>
    <button id="refreshBtn">Refresh Balance</button>
  </div>
</body>
</html>
```

CSS: The Skin, Paint, and Design

What it is: CSS (Cascading Style Sheets) controls how HTML elements are displayed on screen, including layout, colors, and typography.
Analogy: Painting the house, choosing the furniture, and arranging the rooms.

Key Concepts:
Selectors: How you target specific HTML elements to style them.
- Element Selector: Targets all elements of a specific type (e.g., p { color: black; }).
- Class Selector: Targets elements with a specific class attribute. It starts with a dot (e.g., .content { padding: 10px; }).
- ID Selector: Targets a single element with a specific ID. It starts with a hash (e.g., #header { background: blue; }).
The Box Model: Every HTML element is treated as a box. It consists of margins (outer invisible space), borders (the outline), padding (inner space between the border and content), and the actual content itself.
Including CSS: The best practice is to put CSS in an External file (like style.css) and link it in the HTML head.

CSS Example:

```
/* Targeting by ID */
#header {
  background-color: #1a202c;
  color: white;
  text-align: center;
  padding: 20px;
}
/* Targeting by Class */
.content {
  font-family: Arial, sans-serif;
  margin: 20px;
  border: 1px solid #ccc;
}
```

Bootstrap Framework:
Bootstrap is a massive, pre written CSS file. Instead of writing custom CSS for a button from scratch, you just add Bootstrap classes to your HTML like class="btn btn-primary". It saves huge amounts of time, especially for creating grid layouts and responsive designs (making the website look good on both phones and desktops).

JavaScript: The Brain and Behaviors

What it is: A programming language that allows you to implement complex features, logic, and interactivity on web pages.
Analogy: The electricity, plumbing, and smart home systems that make the house actually function and respond to your actions.

Key Concepts:
Variables: Containers for storing data. Use 'const' for values that will never change, and 'let' for values that will change later.
Data Types: Strings (text), Numbers, Booleans (true or false), Arrays (lists of items), and Objects (collections of key and value pairs).
Functions: Blocks of reusable code designed to perform a particular task.
DOM Manipulation: The DOM (Document Object Model) is how JavaScript sees the HTML structure. JavaScript can access and change the DOM. It can read text, change colors, hide elements, or create brand new HTML elements on the fly.
Events: JavaScript can listen for user actions, like mouse clicks, keyboard presses, or form submissions, and trigger specific functions in response.

JS Example:

```
// 1. Select the button and paragraph from the HTML using the DOM
const btn = document.getElementById('refreshBtn');
const balanceText = document.querySelector('.content p');

// 2. Define a function with the logic to run
const updateBalance = () => {
    let newBalance = 5000 + Math.floor(Math.random() * 100);
    balanceText.textContent = 'Your current balance is $' + newBalance + '.';
    balanceText.style.color = 'green';
};

// 3. Attach an event listener to the button to trigger the function on click
btn.addEventListener('click', updateBalance);
```

Assignment - 3
Building a Simple Interactive Webpage

Question 1: HTML Structure
1. Create a new file named index.html.
2. Set up the basic HTML structure with doctype, html, head, and body tags.
3. Inside the head, add a title tag with the text My FinTech App.
4. Inside the body, add a header tag (h1) with the text Welcome to the Dashboard.
5. Add a paragraph tag (p) below the header with an id of statusMessage and text System is ready.
6. Add a button element below the paragraph with an id of actionButton and text Click to Update.

Question 2: CSS Styling(Use custom css or/and bootstrap)
1. Create a new file named style.css and link it in your HTML head section using the link tag.
2. Set the body font family to Arial and the background color to a light gray (e.g., #f4f4f9).
3. Center align the text of the h1 header and change its color to dark blue.
4. Style the button to have a green background, white text, padding of 10px 20px, and remove the default border.

Question 3: JavaScript Interactivity
1. Create a script tag at the bottom of your HTML body (or create a separate script.js file and link it).
2. Select the button using document.getElementById.
3. Add a click event listener to the button.
4. When the button is clicked, change the text of the statusMessage paragraph to System updated successfully!

Drive Link

https://drive.google.com/drive/folders/1x1GX7sgVCRofglkkyyBLxju1AGm8MsFo?usp=sharing

Assignment submission link

https://drive.google.com/drive/folders/1hA5mtnDB2w5YAgTPLNvp_TCCYFDXPepv?usp=sharing

Questions- 07-04-2026

What is **rate limiting**,
What is the difference between **stateless** and **stateful** servers?
What is **CORS**, and why do we need it?
Explain the **request-response lifecycle** in a web server.
What is the difference between **synchronous** and **asynchronous** code on the backend?
What are **sessions vs tokens** for authentication?
Explain **CSRF vs XSS** attacks and how to prevent them.
How does **API versioning** work, and why is it important?

Questions- 08-04-2026

Difference between React and Angular? - Himanshu
What are components in React? - Akshay
Difference between functional and class components? - Payal
What is JSX? - Tejaswa
What are props in React? - Priyanka
What is the state in React? - Ashwin
What is the virtual DOM? - Aryan
What is a React hook? - Shakeel
Difference between controlled and uncontrolled components? - Sachin

Questions- 09-04-2026

What is app.use() in Express? - akshay
What is res.send() vs res.json()? - himanshu
What is MongoDB Atlas? - aryan
What is a connection string? - ashwin

What is indexing in MongoDB? - shakeel
What is validation in Mongoose? - tejaswa
What is request header? - priyanka
What is JWT authentication? - sachin
What is hashing (password)? - payal

Questions : 10-04-2026

What is a React Router? - Aryan
What is environment variable (.env)? - Sachin
What is process.env? -
What is app.listen() in Express? -himashu
What is a port in Node.js?
What is middleware chaining? - akshay
What is res.status()? -
What is ObjectId in MongoDB?
What is lean() in Mongoose? - priyanka
What is toJSON() method? - ashwin
What is modular routing in Express? - payal
What is req.method? - tejaswa
What is req.url? -
What is res.end()? - shakeel
What is $inc operator?

Questions - 13-04-2026

What is the difference between useState and useReducer? - payal
How does the React component lifecycle work (functional components)? - sachin
What is the purpose of useEffect cleanup function? - shakeel
What are controlled vs uncontrolled components? - aryan
What is prop drilling and how can you avoid it? - priyanka
What is React Context API and when should you use it? - ashwin
How does React handle re-rendering? - himanshu
What are keys in React and why are they important? - akshay
What is memoization in React (React.memo, useMemo)? - tejaswa

Assignment - Node.js Basics

Question 1: Creating a Basic Server
1. Create a file named `server.js`.
2. Set up an HTTP server using the built-in `http` module.
3. Make the server listen on port 3000 and respond with "Hello, Node.js!".

Question 2: Asynchronous Code
1. Create a new file called `asyncDemo.js`.
2. Write a script that logs "Start" to the console.
3. Add a `setTimeout` function that logs "Timer Finished" after a 1-second delay.
4. Add `console.log("Middle")`.
3. Add a `setTimeout` function that logs "Timer Finished" after a 2-second delay.
4. Add `console.log("End")` at the bottom and observe the output order when you run it.

Notes: Node.js

What is Node.js?
- It's a Runtime Environment, not a programming language.
- Built on Chrome's V8 JavaScript Engine.
- Allows you to run JavaScript on the backend/server.
- Key characteristics: Asynchronous, Single-threaded, Non-blocking I/O.

When to Use Node.js:
- Good for: Real-time apps (Chat), API servers, Streaming (Netflix), Single Page App backends.
- Bad for: Heavy CPU tasks (Video Editing, complex math, AI) because it is single-threaded and heavy tasks block the whole app.

Node.js Architecture & Event Loop:
- Analogy: Like a bank with one teller (Event Loop) and a back-office team (Worker Pool).
- Call Stack: Tracks where we are in the code (Last In, First Out).
- Node APIs: Handles asynchronous tasks (like setTimeout, database calls, or file reads) in the background.
- Callback Queue: Waiting area for finished async tasks.
- Event Loop: Constantly checks if the Call Stack is empty. If empty, it moves completed tasks from the Callback Queue to the Call Stack.

Modules:
- Code organization by splitting logic into separate files (modules).
- Exporting: module.exports = functionName;
- Importing: const functionName = require('./filename');

Built-in Modules:
1. HTTP: Used to create a web server.
   - Example: Setting up a server that listens on port 3000 and responds to incoming requests.

2. File System (fs): Used to read and write files on the server.
   - fs.writeFile(): Creates or writes to a file.
   - fs.readFile(): Reads the contents of a file.
   - Important detail: Use 'utf8' encoding when reading.
     - Without 'utf8': Node returns a Buffer (binary 0s and 1s, e.g., <Buffer 48 65...>).
     - With 'utf8': Node converts the binary into readable text.

Testing with Jest:
- Testing is crucial for making sure code is reliable before deployment.
- Installation: run `npm install --save-dev jest`.
- Setup: Add `"test": "jest"` to the scripts section of your package.json.
- Syntax:
```
test('adds 1 + 2 to equal 3', () => {
```

```
    expect(add(1, 2)).toBe(3);
  });
```
- Run the tests in the terminal using `npm test`.

Notes - Express.js

What is Express.js?
- A fast, unopinionated, minimalist web application framework for Node.js.
- It provides a robust set of features for web and mobile applications, making it much easier to build servers and APIs compared to using raw Node.js (the built-in `http` module).
- Analogy (Building a House): Plain Node.js is like building a house from raw materials. Express.js is like using a pre-built house frame. It gives you the basic structure (routing, request handling) so you can focus on building the rooms (your features) instead of the foundation.

Basic Project Setup:
1. Initialize the project: `npm init -y`
2. Install Express: `npm install express`

Creating an Express Server:
- Import Express: `const express = require('express');`
- Initialize: `const app = express();`
- Listen: `app.listen(3000, () => console.log('Server running!'));`
- Routing: Determines how an application responds to a client request to a particular endpoint (URI).
  - Syntax: `app.METHOD(PATH, HANDLER)`
  - Example:

```
app.get('/', (req, res) => {
  res.send("Hello Express");
});
```

Middleware:
- Middleware functions are functions that have access to the request object (req), the response object (res), and the next middleware function (`next()`).
- They can execute code, make changes to the req/res objects, end the request-response cycle, or pass control to the next middleware.
- Analogy 1 (Factory Assembly Line): A request comes in and moves through stations. Station 1 logs the request, Station 2 checks authentication. Each does a specific job and passes the request down the line.
- Analogy 2 (Office Security Check):
  - Client: You arriving at the building.
  - Middleware: The security guard checking your ID. If authorized, they let you pass by calling `next()`. If not, they end the request (turn you away).
  - Route Handler: The office you are trying to reach inside the building.

Advanced Middleware & Error Handling:
- Error handlers act as "Safety Nets" for your application.

1. Custom 404 Error Handler:

  - Placed after all your valid routes to catch any request that didn't match.
  - Example:
    app.use((req, res, next) => {
      res.status(404).send("Sorry, that page doesn't exist!");
    });

2. Global Error Handler:
  - A special middleware that takes 4 arguments: `(err, req, res, next)`.
  - Placed at the very bottom of your index.js file.
  - Catches unexpected server errors/crashes and prevents the entire app from going down.
  - Example:
    app.use((err, req, res, next) => {
      console.error(err.stack);
      res.status(500).send('Something broke!');
    });

Testing Express Routes (Jest & Supertest):
- You can test Express APIs without starting a physical server using Supertest.
- Setup: `npm install --save-dev supertest`
- Example:
```
const request = require('supertest');
const app = require('./index');

describe('GET /', () => {
  it('should respond with the homepage', async () => {
    const response = await request(app).get('/');
    expect(response.statusCode).toBe(200);
  });
});
```

Assignment - Express.js

Question 1: Setting up an Express Server
1. Initialize a new Node project (`npm init -y`) and install Express (`npm install express`).
2. Create a file named `app.js`.
3. Import Express and start a server listening on port 3000.
4. When the server starts, log "Express server is running on port 3000" to the console.

Question 2: Creating Routes
1. Add a GET route to the root URL (`/`) that responds with the text "Welcome to the FinTech API!".
2. Add a GET route to `/about` that responds with "This API powers our FinTech application."

Question 3: Handling JSON Data (POST Request)
1. Add the built-in middleware to parse incoming JSON data (`app.use(express.json())`).
2. Create a POST route at `/users`.
3. When a POST request is made to `/users`, the route should read a `name` property from the request body (req.body.name).
4. Respond with a JSON object or string saying: "User [name] has been created successfully!". Test this using Postman.

Notes - API'S AND MONGOOSE

BACKEND ARCHITECTURE AND SETUP
==========

The page talks about building a backend for a library system. The whole system has different parts that work together:

1. Client (React or Postman for testing)
2. Router (handles URL endpoints)
3. Controller (handles logic)
4. Repository (handles database communication)
5. Database (MongoDB stores data)

RESTFUL API ENDPOINTS FOR LIBRARY PROJECT
==========

These are the basic operations needed:

- POST to /api/books - allows adding a new book
- GET to /api/books - shows all books
- GET to /api/books with ID - shows one specific book
- PUT to /api/books with ID - allows updating a book
- DELETE to /api/books with ID - removes a book
- POST to /api/books/upload - lets you upload book cover image

FETCHING DATA WITH QUERY PARAMETERS
==========

Query parameters are extra information added to the URL after the question mark to filter or change the data returned.

Two ways to fetch data:

Fetch API - This is built into the browser, free, no installation needed. You have to manually handle JSON and check errors yourself. Code is a bit longer.

Axios - This is a library you need to install. It automatically handles JSON and errors for you. The code is cleaner and shorter. It also has extra features like interceptors and easy timeout handling.

DETAILED COMPARISON AXIOS VS FETCH
==========

Built-in
- Axios needs to be installed
- Fetch is already in the browser

JSON handling
- Axios automatically uses res.data
- Fetch requires you to use res.json()

Error handling
- Axios handles errors automatically
- Fetch needs you to check response ok manually

Request body
- Axios takes objects directly
- Fetch needs JSON.stringify

Interceptors
- Axios has them available
- Fetch does not have them

Timeout
- Axios is easy to set timeout
- Fetch does not have built-in timeout

Progress tracking
- Axios supports it
- Fetch does not support it

Old browser support
- Axios is better
- Fetch needs polyfill for old browsers

Syntax
- Axios is cleaner and shorter
- Fetch is slightly longer

SENDING DATA TO THE SERVER
==========

POST Request - sending new data

- Use POST method
- Add Content Type header as json
- Use JSON.stringify to send the data
- Examples show sending user data like name and age

PUT Request - updating existing data
- Use PUT method with ID in the URL path
- Also needs Content Type header as json
- Also needs JSON.stringify for the data
- The ID tells which item to update

SETTING UP THE LIBRARY PROJECT

First steps:
1. Create a folder called backend
2. Initialize npm with npm init y
3. Install required packages express mongoose cors dotenv nodemon

These packages do:
- express: the web server framework
- mongoose: talks to MongoDB database
- cors: allows requests from different sources
- dotenv: manages secret settings
- nodemon: restarts server automatically when code changes

FOLDER STRUCTURE FOR THE PROJECT

The project is organized in layers:

backend folder contains:
- config folder for database connection setup
- controllers folder for request and response handling
- models folder for database schema definitions
- repositories folder for database access code
- routes folder for URL endpoints
- index.js file as the main entry point

DATABASE CONNECTION

MongoDB connection code:
- Tries to connect to mongodb at localhost on port 27017
- Database name is libraryDB
- If connection fails it prints an error message and stops

DEFINING WHAT DATA LOOKS LIKE
==========

Book schema defines what information a book record has:
- title: text field, required
- author: text field, required
- isbn: text field, should be unique
- status: text field, default value is Available
- coverImage: text field for image path

IMPLEMENTING THE API
==========

The Repository Pattern separates two concerns:
- Business logic goes in Controller
- Database work goes in Repository

Repository handles the database directly:
- findAll method returns all books from database

Controller handles HTTP requests and responses:
- getAllBooks gets the request from user
- Calls repository to get the data
- Sends data back to user
- If there is an error sends 500 status with error message

Routes map the URLs to controller functions:
- GET to / calls getAllBooks controller function

ADVANCED FEATURES
==========

File Uploads:
- Install multer library for file handling
- Create upload middleware

- POST endpoint for creating books
- POST endpoint for uploading files at /upload
- Checks if file exists before saving

Entry Point Setup:
- Import all required libraries
- Set up express app
- Connect to database
- Use CORS to allow requests from different origins
- Use express.json to process json data
- Serve uploaded files as static files
- Set up the books routes
- Start server on port 5000

RUNNING THE PROJECT
==========

Steps to run:
1. Create uploads folder
2. Run node index.js to start the server

Testing with Postman:
- POST to http://localhost:5000/api/books to add a book
- GET to http://localhost:5000/api/books to see all books
- POST to http://localhost:5000/api/books/upload to upload an image

ASSIGNMENT - API

COMPLETE API FLOW
1. config/db.js - database connection
2. models/Product.js - product schema
3. repositories/productRepository.js - all database methods
4. controllers/productController.js - all controller functions
5. routes/productRoutes.js - all routes with file upload
6. index.js - main entry point and server setup

Database: productDB
Port: 5000

1. Write code to get users from the API with query parameters:
URL: https://api.example.com/users?name=Rahul&age=25

Using Axios with params object

2. Write code to POST a new user with this data:
name: "John"
age: 30

URL: https://api.example.com/users

using Axios

3. Write code to update user with ID 123 to:
name: "UpdatedName"
age: 28

URL: https://api.example.com/users/123

using Axios

4. Write config/db.js code to connect to MongoDB:
- Use productDB database
- Handle connection errors
- Export the connectDB function

5. Write models/Product.js code to create a Product schema with:
- name (required string)
- price (required number)
- sku (unique string)

- status (string with default "In Stock")
- imageUrl (string)
- Export the model

6. Write repositories/productRepository.js with these methods:
- findAll() - returns all products
- findById(id) - returns one product by id
- create(productData) - adds new product
- update(id, productData) - updates product
- delete(id) - deletes product

7. Write controllers/productController.js with these functions:
- getAllProducts(req, res) - get all products
- getProductById(req, res) - get one product
- createProduct(req, res) - create new product
- updateProduct(req, res) - update product
- deleteProduct(req, res) - delete product

All should use the repository and handle errors

8. Write routes/productRoutes.js to map these routes:
- GET / - call getAllProducts
- GET /:id - call getProductById
- POST / - call createProduct
- PUT /:id - call updateProduct
- DELETE /:id - call deleteProduct

9. Add to routes/productRoutes.js:
- POST /upload - use multer to upload single file named "imageUrl"
- Return the file path on success
- Return error if no file uploaded

10. Write index.js with:
- Import all packages and routes
- Connect to database
- Setup CORS
- Setup express.json()
- Setup static files at /uploads
- Setup product routes at /api/products
- Listen on port 5000 and print server running message

11. Write axios code to get products with try-catch error handling:

- Catch request errors
- Catch response errors
- Print error message

12. Write axios code with request interceptor that:
- Adds Authorization header with token "Bearer xyz123"
- Logs the request URL

13. Write axios code to get users by filtering with these params:
- name=Rahul
- age=25
Build the URL string with these parameters

14. For each endpoint write the Postman test steps:

1. POST http://localhost:5000/api/products
   - Add product with name and price in JSON body

2. GET http://localhost:5000/api/products
   - Get all products

3. GET http://localhost:5000/api/products/123
   - Get product with ID 123

4. PUT http://localhost:5000/api/products/123
   - Update product with ID 123

5. DELETE http://localhost:5000/api/products/123
   - Delete product with ID 123

6. POST http://localhost:5000/api/products/upload
   - Upload image file

15. Create a new API's for Student Management:
Do same as Que 14 but for Student instead of Product

Student schema fields:
- name (required)
- email (required, unique)
- rollNumber (required, unique)
- marks (number, default 0)

Endpoints needed in backend and then test by supertest

- POST /api/students - create student
- GET /api/students - get all students
- GET /api/students/:id - get one student
- PUT /api/students/:id - update student
- DELETE /api/students/:id - delete student

NOTES - React.js

SECTION 1: CORE REACT CONCEPTS
==========

1. SINGLE PAGE APPLICATIONS (SPA)

What is it?
- A web app that loads ONE HTML page and dynamically updates content as users interact with it
- Feels like a desktop or mobile app - smooth and fast experience

How it's different from traditional websites:
- Traditional websites: Every click = full page reload from server
- SPA: Initial page load happens once, then only small JSON data is fetched to update parts of the page
- Result: Much faster and smoother user experience

Example: Gmail, Google Maps, Facebook - they don't reload the entire page when you click something

2. COMPONENT-ORIENTED ARCHITECTURE

What is it?
- React is all about breaking down your UI into small, reusable pieces called COMPONENTS
- Think of it like LEGO bricks - you build complex structures from simple, individual pieces

How it works:
- Each component is self-contained and can be reused
- Complex UIs are made by combining simple components together

Example structure:
- App Component (entire page)
  ├── Header Component (top navigation)
  ├── Sidebar Component (left side menu)
  └── Content Component (main area)

Benefits:
- Easy to manage because each component does one thing
- Can reuse components across different pages
- Easier to find and fix bugs

3. JSX, BABEL, AND WEBPACK

These three tools work together to make React possible:

JSX (JavaScript XML):
- A syntax that lets you write HTML-like code directly in JavaScript
- Makes UI code much more readable and intuitive
- Example: <h1>Hello World</h1> instead of createElement('h1', 'Hello World')

Babel:
- A JavaScript compiler (translator)
- Converts modern JavaScript and JSX into older JavaScript that ALL browsers understand
- Makes sure your code runs everywhere, even in older browsers

Webpack:
- A module bundler
- Takes all your files (JavaScript, CSS, images)
- Combines them into optimized files that are fast to download
- Makes sure dependencies are loaded in the right order

==========
SECTION 2: REACT LAB - BUILDING YOUR FIRST APP
==========

1. CHOOSING A TOOL: VITE vs CREATE REACT APP

There are different tools to start a React project. Let's compare:

VITE (Modern - RECOMMENDED):
- Setup: npm create vite@latest
- Speed: EXTREMELY FAST development
- Hot Module Replacement (HMR): Near-instant updates while coding
- Status: Actively developed, recommended by React community
- Best for: New React projects, faster workflow

CREATE REACT APP (Legacy):
- Setup: npx create-react-app my-app
- Speed: Slower (bundles all code before showing anything)
- HMR: Slow updates
- Status: No longer recommended for new projects
- Issues: Tooling is outdated

DECISION: WE WILL USE VITE because it's faster, modern, and gives a better learning experience.

2. HOW TO CREATE YOUR FIRST REACT SPA WITH VITE

STEP 1: Set up with Vite
Command: npm create vite@latest

Vite will ask you:
- Project name? >> Type: frontend
- Select a framework? >> Choose: React
- Select a variant? >> Choose: JavaScript

Then run:
```
cd frontend
npm install
npm run dev
```

Your app is now running! But to test it on your phone on the same network:
```
npm run dev -- --host
```

STEP 2: Create Reusable Components

You'll create three main components: Header, Content, Footer

FILE: src/Header.jsx

```
function Header(props) {
  return (
    <header style={{ backgroundColor: '#0d1117', color: 'white', padding: '1rem' }}>
      <h1>{props.title}</h1>
    </header>
  );
}
export default Header;
```

What's happening here?
- props.title: Gets a title passed from the parent component
- Style object: Inline styling with a dark background and white text
- Export: Makes this component available to other files

FILE: src/Content.jsx

```
function Content() {
  return (
    <main style={{ padding: '1rem' }}>
      <p>Welcome to MERN stack.</p>
    </main>
  );
}
```

What's happening here?
- Simple component that displays a message
- <main> tag is for semantic HTML (tells search engines this is the main content)

FILE: src/Footer.jsx

```
function Footer() {
  const currentYear = new Date().getFullYear();
  return (
    <footer style={{ padding: '1rem', borderTop: '1px solid #ccc' }}>
      <p>© {currentYear} FinTech Course. All Rights Reserved.</p>
    </footer>
  );
}
```

What's happening here?
- Gets current year automatically with new Date().getFullYear()
- Uses curly braces {} to insert JavaScript into JSX
- Shows how to dynamically update content

STEP 3: Assemble the App - Using Props

FILE: src/App.jsx

```
import './App.css';
import Header from './Header.jsx';

function App() {
  return (
    <div className="App">
```

```
      <Header title="MERN Stack for FinTech" />
    </div>
  );
}
```

What's happening here?
- Imports the Header component
- Passes a "title" prop to Header with value "MERN Stack for FinTech"
- Parent component (App) passes data DOWN to child component (Header)
- Header receives it as props.title and displays it

Example of Props:
- Props = properties passed from parent to child
- Like function arguments, but for components
- One-way flow: Parent → Child (not the other way around)

STEP 4: Add Styling

FILE: src/App.css

```
.App {
  text-align: center;
}

main p {
  font-size: 1.2rem;
  color: #333;
}
```

How to use:
- Create CSS files alongside your components
- Import them in App.jsx: import './App.css';
- Use className instead of class in JSX (because class is a reserved word in JavaScript)

Example of what you'll see:
- Header with dark background and white title
- Main content with centered text
- Footer with copyright year (automatically updates next year!)

==========

KEY TAKEAWAYS
==========

1. SPAs are fast and smooth - like mobile apps
2. Components are reusable building blocks of your UI
3. Props let parent components pass data to child components
4. Vite is the modern, fast way to build React projects
5. Styling can be done inline or in separate CSS files
6. Always use className, not class, in JSX
7. Export components so other files can use them
8. Import components before you use them in other files

==========

ASSIGNMENT: React.js

EXERCISE 1: Set Up Vite React Project
==========

Create a new React app using Vite:

1. Run: npm create vite@latest my-react-app

2. Select React and JavaScript
3. Navigate to folder: cd my-react-app
4. Install dependencies: npm install
5. Start development server: npm run dev
6. Open in browser and verify default app is running

Deliverable: Screenshot of running Vite app in browser

==========
EXERCISE 2: Build Header Component
==========

Create file: src/Header.jsx

Code requirements:
- Function component named Header
- Accept props.title
- Display title in <h1> tag
- Style with backgroundColor: '#0d1117' and color: 'white'
- Add padding: '1rem'
- Export the component

Expected Result:
When used as <Header title="My App" />, should display "My App" in white text on dark background

Full Code Template to Complete:

```
function Header(props) {
  return (
    <header style={{ _____, _____, _____ }}>
      <h1>_____</h1>
    </header>
  );
```

```
}
export default _____;
```

==========
EXERCISE 3: Build Content Component
==========

Create file: src/Content.jsx

Code requirements:
- Function component named Content
- Display inside <main> tag
- Show a paragraph with text: "Welcome to MERN Stack"
- Add padding: '1rem'
- Optional: Add some descriptive text below the welcome message
- Export the component

Expected Result:
Display main content with padding and welcome message

Full Code Template to Complete:

```
function Content() {
  return (
    <main style={{ _____ }}>
      <p>_____</p>
      <p>You can add more content here...</p>
    </main>
  );
}
export default _____;
```

==========
EXERCISE 4: Build Footer Component
==========

Create file: src/Footer.jsx

Code requirements:
- Function component named Footer
- Get current year dynamically using: new Date().getFullYear()
- Display in <footer> tag

- Show copyright text with current year: © 2026 Company. All Rights Reserved.
- Add styling: padding: '1rem', borderTop: '1px solid #ccc'
- Export the component

Expected Result:
Footer that automatically shows current year (2026, and will update to 2027 next year)

Full Code Template to Complete:

```
function Footer() {
  const currentYear = _____;
  return (
    <footer style={{ _____, _____ }}>
      <p>© {_____} Company. All Rights Reserved.</p>
    </footer>
  );
}
export default _____;
```

==========
EXERCISE 5: Assemble Components in App.jsx
==========

Modify file: src/App.jsx

Code requirements:
- Import Header.jsx
- Import Content.jsx
- Import Footer.jsx
- Import './App.css'
- Remove all default Vite code
- Create layout with all three components:
  - Header with title prop: "MERN Stack for FinTech"
  - Content component
  - Footer component
- Wrap in a div with className="App"

Expected Result:
App displaying Header at top, Content in middle, Footer at bottom

Full Code Template to Complete:

```
import '_____';
```

```
import _____ from '_____';
import _____ from '_____';
import _____ from '_____';

function App() {
  return (
    <div className="App">
      <Header title="_____" />
      <_____ />
      <_____ />
    </div>
  );
}

export default App;
```

==========

EXERCISE 6: Add CSS Styling

==========

Create/Modify file: src/App.css

Code requirements:
- Center the .App container (text-align: center)
- Style main paragraph with:
  - Font size: 1.2rem
  - Color: #333
  - Add padding to main content area
- Add margin between components
- Optional: Add a max-width container, add hover effects

Example CSS needed:
- .App { text-align: center; }
- main { padding: 2rem; }
- main p { font-size: 1.2rem; color: #333; }

Test: Verify App.jsx imports this CSS file with: import './App.css';

==========

EXERCISE 7: Use Props - Add Subtitle

==========

Modify file: src/Header.jsx

Current code:
```
function Header(props) {
  return (
    <header style={{ backgroundColor: '#0d1117', color: 'white', padding: '1rem' }}>
      <h1>{props.title}</h1>
    </header>
  );
}
```

Changes needed:
- Accept a new prop: props.subtitle
- Display subtitle below title in an <h2> or <p> tag
- Style subtitle with color: '#999' or '#aaa' for gray color

New usage in App.jsx:
```
<Header
  title="MERN Stack for FinTech"
  subtitle="Learn Full Stack Development"
/>
```

Updated Header Code:
```
function Header(props) {
  return (
    <header style={{ backgroundColor: '#0d1117', color: 'white', padding: '1rem' }}>
      <h1>{props.title}</h1>
      <p style={{ color: '_________' }}>_________</p>
    </header>
  );
}
```

==========
EXERCISE 8: Create a Banner Component
==========

Create file: src/Banner.jsx

Code requirements:
- Function component named Banner
- Accept props:
  - props.message (string)
  - props.bgColor (background color, default 'lightblue')

  - props.textColor (text color, default 'black')
- Display banner as a with the message
- Apply background and text colors
- Add padding and margin
- Export component

Usage example:
<Banner message="Welcome to React!" bgColor="lightgreen" textColor="darkgreen" />

Expected Result:
Colorful banner component with custom colors

Template:
```
function Banner(props) {
  return (
    <div style={{
      backgroundColor: props.bgColor || '_________',
      color: props.textColor || '_________',
      padding: '_________',
      margin: '_________',
      textAlign: 'center',
      borderRadius: '5px'
    }}>
      <p>{___________}</p>
    </div>
  );
}
export default ___________;
```


==========
EXERCISE 9: Props with Multiple Config
==========

Create file: src/Card.jsx

Code requirements:
- Function component named Card
- Accept props:
  - props.title
  - props.description
  - props.imageUrl
- Display as a card with:

  - Image at top (use)
  - Title in <h3>
  - Description in
  - Border: '1px solid #ddd'
  - Padding: '1rem'
  - Border radius: '8px'
- Export component

Usage in App.jsx:
<Card
  title="React Basics"
  description="Learn the fundamentals of React"
  imageUrl="./images/react-logo.png"
/>

Note: Create placeholder imageUrl or use a web image URL

Template:
function Card(props) {
  return (
    <div style={{ border: '_____', padding: '_____', borderRadius: '_____' }}>

      <h3>{_____}</h3>
      {_____}

  );
}
export default _____;

==========
EXERCISE 10: List Component with Loop
==========

Create file: src/CourseList.jsx

Code requirements:
- Function component named CourseList
- Accept props.courses (array of course objects)
- Each course object has: { id, name, duration }
- Loop through array using .map()
- Display each course as an <li> in a
- Show course name and duration
- Export component

Usage in App.jsx:
```
const courses = [
  { id: 1, name: 'React Basics', duration: '4 weeks' },
  { id: 2, name: 'Advanced React', duration: '6 weeks' },
  { id: 3, name: 'Node.js Backend', duration: '5 weeks' }
];

<CourseList courses={courses} />
```

Expected Result:
Unordered list showing all courses with their durations

Template:
```
function CourseList(props) {
  return (
    <ul>
      {props.courses.map((course) => (
        <li key={_____________}>
          {_____________} - {_____________}
        </li>
      ))}
    </ul>
  );
}
export default _____________;
```

==========
EXERCISE 11: Component with State Prep
==========

Create file: src/Welcome.jsx

Code requirements:
- Function component named Welcome
- Accept props.userName
- Display personalized welcome message
- If props.userName is provided, show: "Welcome, {userName}!"
- If props.userName is not provided, show: "Welcome, Guest!"
- Use ternary operator: props.userName ? '...' : '...'

Usage:
```
<Welcome userName="Ani" />
```

<Welcome />

Expected Result:
First shows "Welcome, Ani!"
Second shows "Welcome, Guest!"

Template:
```
function Welcome(props) {
  return (
    <div>
      <h2>
        {props.userName ? _____ : '_____'}
      </h2>
    </div>
  );
}
export default _____;
```

==========
EXERCISE 12: Conditional Rendering
==========

Create file: src/Status.jsx

Code requirements:
- Function component named Status
- Accept props.isOnline (boolean - true or false)
- If isOnline is true: Show "🟢 Online" with green color
- If isOnline is false: Show "🔴 Offline" with red color
- Use conditional ? : operator or if/else
- Add padding and styling

Usage:
```
<Status isOnline={true} />    // shows green online
<Status isOnline={false} />   // shows red offline
```

Expected Result:
Status indicator that changes color based on isOnline prop

Template:
```
function Status(props) {
  return (
    <div style={{ padding: '1rem' }}>
```

```
      {props.isOnline ? (
        <p style={{ color: '_________' }}>🟢 __________</p>
      ) : (
        <p style={{ color: '_________' }}>🔴 __________</p>
      )}
    </div>
  );
}
export default __________;
```

==========
FINAL CHALLENGE: Complete Mini App
==========

Integrate all exercises into one working app:

Requirements:
1. App.jsx imports and displays:
   - Header with title and subtitle
   - Banner component
   - Content component
   - CourseList component (with sample courses array)
   - Card component (at least one)
   - Welcome component
   - Status component (toggle online/offline)
   - Footer

2. All components should be styled and properly formatted

3. At least 3 different props passed between components

4. App should run without console errors

5. All components should display correctly in browser

Deliverable:
- Screenshot of completed app
- Run: npm run dev and open in browser
- Verify all components render with no errors
- Code files: App.jsx, App.css, Header.jsx, Content.jsx, Footer.jsx, Banner.jsx, Card.jsx, CourseList.jsx, Welcome.jsx, Status.jsx

==========
SUBMISSION CHECKLIST
==========

□ Exercise 1: Vite project created and running
□ Exercise 2: Header.jsx created with props
□ Exercise 3: Content.jsx created
□ Exercise 4: Footer.jsx with dynamic year
□ Exercise 5: App.jsx assembles all components
□ Exercise 6: App.css styling applied
□ Exercise 7: Header props enhanced with subtitle
□ Exercise 8: Banner.jsx created with multiple props
□ Exercise 9: Card.jsx created with image and content
□ Exercise 10: CourseList.jsx displays array of courses
□ Exercise 11: Welcome.jsx with conditional message
□ Exercise 12: Status.jsx with color-coded indicator
□ Final Challenge: All components working together
□ No console errors in browser
□ npm run dev runs successfully
□ All files properly exported/imported

==========

Notes - Routers & Hooks/classes

1. React Router: The GPS of your App
- Traditional HTML (MPA): Click link -> Server request -> Full reload (slow)
- React Router (SPA): Click link -> JS intercepts -> Swap component (instant)
- Required for SPAs to have different URLs and browser history without reload.
- Alternatives: Conditional rendering (messy), other libraries like TanStack Router, Wouter.
- Page Components: Create Home and About components.
- Implement in App.jsx: Import BrowserRouter, Routes, Route, Link; wrap with BrowserRouter, add nav with Links, Routes with Route elements.
- Entry point: Keep React.StrictMode.
- Components:
  - BrowserRouter: Container, wraps everything.
  - Routes: Groups routes, ensures one shows.
  - Route: Defines path and component.
  - Link: Navigation, changes URL without reload.

2. Classes vs. Hooks: The Evolution
- React originally used Class Components, then Hooks in 2018 for Functional Components.
- Comparison:
  - Syntax: class App extends React.Component vs function App()
  - State: this.state = {count:0} vs const [count, setCount] = useState(0)
  - Lifecycle: componentDidMount() vs useEffect()
  - Complexity: High vs Low
- Code examples for Counter.

3. Essential Hooks Toolkit
- useState: Memory, stores changing data (input text, counter, toggle)
- useEffect: Side effects, runs after render (API fetch, timer, update title)
- useContext: Global data, avoids prop drilling (user login, theme)
- useRef: DOM access, reference elements (focus input)
- useParams: URL data, reads dynamic parts (id from /product/123)

4. Examples
- Digital Clock: useState for time, useEffect for timer update and cleanup.
- Window Size Tracker: useState for width, useEffect for resize listener and cleanup.

5. The "Like" Counter (useState)
- useState(42), button onClick setLikes(likes+1)

6. Why does it reset on refresh?
- State is in RAM, cleared on refresh.
- Solution: Use localStorage.
- Initialize useState with localStorage.getItem or 0

- useEffect to setItem when likes change.

7. Why did we need these Hooks?
- For state management: Variables don't update UI, classes are complex; useState hooks into variable.
- For side effects: Manual runs every render; useEffect runs only on change.
- Flow: User clicks -> useState updates UI & memory -> useEffect detects change & saves.

Assignment - Routers & Hooks

1. Create two simple page components: Home and About. Each should return an h2 with the page name.
2. In App.jsx, set up React Router using BrowserRouter, Routes, Route, and Link to navigate between Home (/) and About (/about) pages.
3. Create a LikeCounter component using useState that starts at 0 and increments on button click.
4. Modify the LikeCounter to persist the likes count in localStorage using useEffect, so it doesn't reset on refresh.

Notes - Library App Frontend

1. Setup & Dependencies
- Create the frontend application using Vite, a modern build tool for React that provides fast development and optimized production builds.
- Install necessary dependencies: axios for making HTTP requests to the API, react-router-dom for handling client-side routing, and bootstrap for responsive CSS styling.
- In main.jsx, import Bootstrap's CSS file after the React imports to apply the styles globally across the app.

2. Book List Component
- Create a new file components/BookList.jsx to handle fetching and displaying the list of books from the API.
- Import React, useState, useEffect from 'react', and axios.
- Initialize state: const [books, setBooks] = useState([]); to store the array of books.
- Use useEffect with an empty dependency array to fetch books once when the component mounts: useEffect(() => { fetchBooks(); }, []);
- Define fetchBooks as an async function: try { const res = await axios.get('http://localhost:5000/api/books'); setBooks(res.data); } catch (err) { console.error("Error fetching books:", err); }
- Define handleDelete as an async function taking id: if (window.confirm('Are you sure you want to delete this book?')) { try { await axios.delete(`http://localhost:5000/api/books/${id}`); fetchBooks(); } catch (err) { console.error("Error deleting book:", err); } }
- In the return JSX: div with className="container mt-4", h2 "Library Books", div with className="row", map over books array to create div with className="col-md-4 mb-4" for each book.
- Inside each book div: div with className="card h-100 shadow-sm", conditional rendering of img if book.coverImage exists with src={`http://localhost:5000/${book.coverImage}`}, alt={book.title}, style height 250px objectFit cover.
- Then div className="card-body", h5 className="card-title" {book.title}, h6 className="card-subtitle mb-2 text-muted" "By {book.author}", p className="card-text small" "ISBN: {book.isbn}", button className="btn btn-danger btn-sm" onClick={() => handleDelete(book._id)} "Delete".

3. Add Book Form (with File Upload)
- Create components/AddBook.jsx to handle the form for adding new books, including file upload for cover images.
- Import React, useState from 'react', axios, and useNavigate from 'react-router-dom'.
- State variables: const [formData, setFormData] = useState({ title: '', author: '', isbn: '' }); const [file, setFile] = useState(null);
- const navigate = useNavigate(); to redirect after successful submission.
- handleChange function: (e) => { setFormData({ ...formData, [e.target.name]: e.target.value }); }
- handleFileChange function: (e) => { setFile(e.target.files[0]); }

- handleSubmit async function: (e) => { e.preventDefault(); try { const data = new FormData(); data.append('title', formData.title); data.append('author', formData.author); data.append('isbn', formData.isbn); if (file) { data.append('coverImage', file); } await axios.post('http://localhost:5000/api/books/add', data); navigate('/'); } catch (err) { alert('Error adding book'); console.error(err); } }
- JSX structure: div className="container mt-4", div className="card shadow p-4" with style maxWidth 600px margin 0 auto, h2 "Add New Book", form onSubmit={handleSubmit}.
- Inside form: div className="mb-3" with label "Title" and input type="text" name="title" className="form-control" onChange={handleChange} required.
- Similarly for author and isbn.
- div className="mb-3" label "Cover Image" input type="file" className="form-control" onChange={handleFileChange}.
- button type="submit" className="btn btn-primary w-100" "Add Book".

4. Wiring it up (App.jsx)
- In App.jsx, set up the routing and navigation for the app.
- Import BrowserRouter, Routes, Route, Link from 'react-router-dom', and BookList and AddBook components.
- Wrap the entire app in BrowserRouter.
- Create a navbar: nav className="navbar navbar-expand-lg navbar-dark bg-dark", inside div className="container", Link className="navbar-brand" to="/" "Library App", div with Link className="btn btn-outline-light me-2" to="/" "Home", Link className="btn btn-primary" to="/add" "Add Book".
- Then Routes component with Route path="/" element={<BookList />}, Route path="/add" element={<AddBook />}.

5. Final UI & Integration Checklist
- The final UI shows a clean interface with a dark navbar, book cards in a grid layout, and a centered form for adding books.
- Key points for integrating the React frontend with a Node/Express backend:
  - CORS (Cross-Origin Resource Sharing): Since the frontend runs on port 5173 (Vite default) and backend on 5000, the backend must allow cross-origin requests. Add app.use(cors()) in the Express server index.js file.
  - API Base URL: In development, hardcode 'http://localhost:5000' for API calls. In production, use environment variables like import.meta.env.VITE_API_URL to make it configurable.
  - The useEffect Hook: Place API calls for fetching initial data inside useEffect with an empty dependency array [] to ensure they run only once on component mount, preventing infinite re-rendering loops.
  - Handling File Uploads: Files cannot be sent as JSON. Use the FormData object to create multipart/form-data requests. Append text fields and the file, then POST. Axios automatically sets the correct Content-Type header when it detects FormData.
  - Serving Static Images: The frontend's img src attributes point to the backend's URL, e.g., 'http://localhost:5000/uploads/image.jpg'. The backend must serve the uploads folder statically using app.use(express.static('uploads')) in Express.

Assignment - Country App Frontend

1. Set up a new Vite React project and install axios, react-router-dom, and bootstrap. Add Bootstrap CSS import in main.jsx.

2. Create a components/CountryList.jsx component that fetches countries from 'http://localhost:5000/api/countries' using axios and useState/useEffect.
Display them in Bootstrap cards showing name, capital, and flag image if available. Include a delete button that confirms and deletes via API.

3. Create a components/AddCountry.jsx component with a form for country name, capital, and flag image upload.
Use FormData to submit to 'http://localhost:5000/api/countries/add' and navigate back to home on success.

4. In App.jsx, set up BrowserRouter with routes for '/' (CountryList) and '/add' (AddCountry). Include a navbar with links to Home and Add Country.

Firebase Migration & Deployment Guide (Documentation)

This document outlines the complete transition from a Node.js + Express + MongoDB backend to a fully Firebase-powered architecture, along with image hosting (Cloudinary option) and deployment steps.

---

1. Architecture Change Overview

We are **removing the backend completely**:

- Node.js + Express ⟶ Removed
- MongoDB ⟶ Removed

Firebase will handle everything:

- Authentication ⟶ User login/signup
- Firestore ⟶ Database (products, users)
- Storage ⟶ Product images

The frontend (React) will now directly communicate with Firebase.

---

2. Firebase Project Setup

Step 1: Create Project

- Go to Firebase Console
- Click **Create a project**

Step 2: Enable Authentication

- Navigate: **Build** → **Authentication** → **Get Started**
- Go to **Sign-in method**
- Enable:
    - Email/Password
- Click **Save**

Step 3: Enable Firestore Database

- Navigate: **Build** → **Firestore Database**
- Click **Create Database**
- Select:
    - **Start in Test Mode**

Step 4: Enable Storage

- Navigate: **Build** → **Storage**
- Click **Get Started**
- Select:
    - **Start in Test Mode**

Step 5: Get Firebase Config

- Go to **Project Settings (⚙️ icon)**
- Under **Your apps**
    - Click **Web (</>)**
- Copy:

```
const firebaseConfig = {
  apiKey: "...",
  authDomain: "...",
  projectId: "...",
  storageBucket: "...",
  messagingSenderId: "...",
  appId: "..."
};
```

Paste this into:

/frontend/src/firebase.js

Step 6: Install Firebase SDK

Run inside frontend folder:

npm install firebase

---

3. Replace Backend with Firebase SDK

Create a new file:

/frontend/src/api/productApi.js

Replace all backend API calls with Firebase logic:

- Firestore → Products data
- Firebase Storage → Images

- Firebase Auth → Users

---

4. Component Updates

Navbar

- Remove `localStorage` token logic
- Use Firebase Auth state:
    - `onAuthStateChanged()`

Product List

- Update image source:
    - Use Firebase Storage URL

Add Product Modal

- Send **plain JavaScript objects**
- Do NOT use FormData

Add Product Page

- Store file in state
- Submit as object
- Upload image via Firebase Storage

---

5. Final Result

Frontend directly talks to Firebase
No backend server required
No MongoDB needed

You can safely shut down:

- Express server
- MongoDB

---

6. Alternative: Cloudinary (Image Hosting)

Cloudinary is a **free alternative** for image uploads.

Step 1: Setup Cloudinary

1. Create account
2. From Dashboard:
    - Copy **Cloud Name**
3. Go to:
    - Settings → Upload tab
4. Scroll to:
    - **Upload Presets → Add Upload Preset**
5. Change:
    - Signing Mode → **Unsigned**
6. Copy:
    - Upload Preset Name
7. Click **Save**

Step 2: Update productApi.js

Replace Firebase Storage upload with:

```
const uploadImage = async (file) => {
  const data = new FormData();
  data.append("file", file);
  data.append("upload_preset", "YOUR_UPLOAD_PRESET");

  const res = await fetch(
    "https://api.cloudinary.com/v1_1/YOUR_CLOUD_NAME/image/upload",
    {
      method: "POST",
      body: data
    }
  );

  const result = await res.json();
  return result.secure_url;
};
```

Replace:

- `YOUR_UPLOAD_PRESET`
- `YOUR_CLOUD_NAME`

---

7. Deploy to Firebase Hosting

Step 1: Install Firebase CLI

npm install -g firebase-tools

Step 2: Login

firebase login

Step 3: Initialize Hosting

firebase init hosting

Answer prompts:

- Use existing project → (move by arrow key then press enter)
- Select project → `electronic-product-ce565`
- Public directory → `dist` (if project created by vite(react) then- dist)
- Single Page App → y
- GitHub deploy → N

Step 4: Build React App

npm run build

This creates:

/dist

Step 5: Deploy

firebase deploy

You'll get a live URL like:

https://electronic-product-ce565.web.app

Future Updates

Whenever you change code:

npm run build
firebase deploy

Summary

- Firebase replaces backend completely
- Firestore = Database
- Auth = Login system
- Storage / Cloudinary = Image hosting
- React talks directly to Firebase
- Deployment via Firebase Hosting

If anything breaks during setup (auth, upload, or deploy), debug step-by-step—most issues come from:

- Wrong config keys

- Storage rules
- Incorrect upload preset

MERN Project Deployment on AWS (Beginner Guide)

Goal

- Backend → EC2
- Frontend → S3
- Database → MongoDB Atlas

PART 1: AWS Setup

1. AWS Account Login

- AWS console open karo
- Region select: **Mumbai (ap-south-1)**

PART 2: EC2 Backend Deployment

2. EC2 Instance Launch

- Name: `mern-backend`
- OS: Amazon Linux / Ubuntu
- Instance: **t2.micro (Free Tier)**
- Key Pair: `mern-key.pem` download karo

---

3. Security Group Setup

Inbound rules:

- SSH → 22
- HTTP → 80
- Custom TCP → **5000**

---

4. Connect to EC2

ssh -i "mern-key.pem" ec2-user@<PUBLIC-IP>

---

5. Install Node.js

sudo dnf update -y
sudo dnf install nodejs -y

---

6. Upload Backend (Without Git)

Local PC se:

scp -i mern-key.pem backend.zip ec2-user@<PUBLIC-IP>:/home/ec2-user

---

7. Run Backend

unzip backend.zip
cd backend
npm install
node index.js

PART 3: MongoDB Atlas

8. Setup Database

- Free cluster create karo

- User create karo
- Password note karo

---

9. Network Access

Add IP:

0.0.0.0/0

---

10. Connection String

mongodb+srv://username:password@cluster.mongodb.net/mernDB?retryWrites=true&w=majority

---

11. Add in .env

MONGO_URI=your_connection_string
PORT=5000

PART 4: Backend Fixes

12. Listen on all IPs

app.listen(5000, "0.0.0.0");

---

13. Enable CORS

app.use(cors({ origin: "*" }));

PART 5: Run Backend Permanently

14. Install PM2

sudo npm install -g pm2

---

15. Start App

pm2 start index.js
pm2 save
pm2 startup

(copy & run given sudo command)

PART 6: Frontend Deployment (S3)

16. Build React App

npm run build

---

17. Upload to S3

- Bucket create karo
- Public access allow karo
- Upload **dist/build ke andar ke files**

---

18. Static Hosting Enable

- Index: `index.html`
- Error: `index.html`

PART 7: Connect Frontend + Backend

19. API URL Set

const API = "http://<EC2-IP>:5000";

---

20. Rebuild

npm run build

PART 8: Testing

- Backend:

http://<EC2-IP>:5000

- Frontend:
  → S3 Static URL

COMMON ERRORS

404 on refresh

Fix:

- S3 error document = `index.html`

MongoDB not connecting

Fix:

- IP whitelist `0.0.0.0/0`

Permission denied (npm)

Fix:

sudo npm install -g pm2

Server not responding

Fix:

- Check port 5000 open
- Check server running

PART 9: Stop Everything (Cost Control)

EC2 Stop

- Instance → Stop

S3

- Optional delete

MongoDB

- Free tier

COST

- EC2 (t3.micro) → Free Tier
- S3 → minimal
- Atlas → free

Total ≈ ₹0 (within limits)

IMPORTANT TIPS

- `.env` upload mat karo
- `node_modules` zip mat karo
- Key file safe rakho
- EC2 stop karna na bhoolo
